Semantic Alignment and Risk Prediction in Software Pull Requests

Project Alias:

PRISM: Pull Request Intelligence for Semantic Mapping

An Encoder-Based Framework for Jira–Pull Request Semantic Alignment and Risk Prediction on a Controlled Synthetic Benchmark

Course No.: S1-26_SSWTZG628T

Course Title: Dissertation

Dissertation Done By:

Student Name: Bhagyashri Kiran Bhat

BITS ID: 2022wc15160

Degree Program: MTech - Integrated Software Systems - WIPRO TECHNOLOGIES (WASE)

Research Area: Machine Learning

Project carried out at: Wipro Technologies, Pune



BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI

VIDYA VIHAR, PILANI, RAJASTHAN - 333031

Work Integrated Learning Programmes (WILP) Division

October 2026

Contents

1. Broad Area of Work 3
2. Background 3
3. Objectives 5
4. Scope of Work 5
5. Plan of Work 7
6. Literature References 8
7. Particulars of the Supervisor and Examiner 9
8. Remarks of the Supervisor 9

1. Broad Area of Work

PRISM (Pull Request Intelligence for Semantic Mapping) lies at the intersection of Natural Language Processing (NLP), Machine Learning, and Software Engineering Analytics. It develops an encoder-based framework that measures how well a GitHub pull request (PR) aligns with its linked Jira issue and estimates the risk of the change, without using generative large language models (LLMs). All experiments are carried out on a controlled synthetic benchmark with known ground truth. The broad areas of work include:

- Semantic alignment: encoding Jira and PR text over several PR fields with a single Sentence-BERT (SBERT) bi-encoder, bge-base-en-v1.5, selected for its strong results on the Massive Text Embedding Benchmark (MTEB).
- Pair re-scoring and domain adaptation: re-scoring Jira–PR pairs with a cross-encoder, and fine-tuning the bi-encoder with Multiple Negatives Ranking Loss and hard negatives.
- Hybrid lexical and structural signals: BM25 and TF-IDF scores, identifier and module-name overlap, and matching between the Jira module and the changed file paths and branch name.
- Calibrated alignment classification: combining semantic, lexical, and structural signals in a calibrated classifier whose decision threshold is chosen on validation data, replacing a fixed similarity cut-off.
- Risk prediction: a leakage-free Random Forest compared with logistic regression and gradient-boosting baselines, explained with SHAP.
- Experimental evaluation: a seeded synthetic benchmark with paraphrased text, partially related negatives, noise, explicit ground-truth labels, and difficulty levels, together with a clear statement of its limitations.

2. Background

Software development teams commonly describe requirements, defects, and improvements in issue-tracking systems such as Jira, while pull requests record the corresponding implementation changes. A link between a Jira issue and a PR provides traceability, but it does not show whether the PR actually addresses the issue. Reviewers must also decide which PRs need closer attention, a decision usually made informally from change size and familiarity with the code.

Sentence-BERT (SBERT) and related sentence encoders map text to dense vectors whose cosine similarity reflects semantic closeness, and transformer models have been used for issue-to-commit trace-link recovery.

PRISM addresses a six-stage pipeline (Table 1) built only from encoder models and classical machine learning. A redesigned generator produces harder, labelled synthetic data. Alignment is scored from several PR fields using the bge-base-en-v1.5 bi-encoder, complemented by lexical and structural features and cross-encoder re-scoring, and these signals are combined by a calibrated classifier

instead of a fixed threshold. Risk is predicted from observable change characteristics by a Random Forest trained on labels drawn from a hidden probabilistic model, so the classifier cannot simply reproduce the labelling rule.

Table 1: PRISM pipeline stages and selected techniques

Stage	Model / technique	Output
1. Synthetic data generation	Seeded generator with multiple paraphrase templates and synonyms, partially related and unrelated negatives, text and label noise, hidden probabilistic risk model, three difficulty levels	Jira tickets and PRs with is_aligned, misalignment type, and is_high_risk labels
2. Multi-field encoding	SBERT bi-encoder bge-base-en-v1.5 over PR title, description, code-change summary, and file paths; optional fine-tuning with Multiple Negatives Ranking Loss	Per-field cosine similarity scores
3. Lexical and structural features	BM25 and TF-IDF cosine; identifier and module-name overlap; Jira module to file-path match; branch-name overlap	Lexical and structural feature vector
4. Pair re-scoring	Cross-encoder (for example, cross-encoder/stsb-roberta-base or cross-encoder/ms-marco-MiniLM-L-6-v2) reading the Jira and PR text jointly	Cross-encoder relevance score per pair
5. Alignment classification	Logistic regression and gradient boosting over stages 2–4, probability calibration, threshold selected on validation data	Alignment probability and aligned / misaligned label
6. Risk prediction	Random Forest with class weighting, compared with logistic regression and gradient boosting; SHAP explanations	High-risk probability with feature-level explanations

Generative LLMs are deliberately excluded. bge-base-en-v1.5 was chosen over the prototype's all-MiniLM-L6-v2 because it scores higher on the MTEB semantic-similarity and retrieval tasks while remaining small (about 110 million parameters). It runs on a CPU or a modest GPU, produces

deterministic scores, and are cheaper to evaluate and explain than prompt-based generation. The cross-encoder is more accurate than a bi-encoder for scoring a given pair because it reads both texts together, while the bi-encoder remains useful for ranking all candidate tickets for a PR.

The project aims to support review prioritization and improve issue-to-PR traceability. PRISM is intended to assist reviewers rather than replace code review, and results on synthetic data are treated as evidence of method behaviour under controlled conditions, not as a guarantee of performance on real projects.

3. Objectives

Review research on sentence embeddings, bi-encoders and cross-encoders, issue-to-code traceability, just-in-time change-risk prediction, and synthetic benchmark design, and analyse the limitations of the initial PRISM prototype.
Redesign the synthetic data generator with multiple paraphrase templates and synonyms, partially related negatives (same module, different task), unrelated negatives, text and label noise, an explicit is_aligned column with misalignment type, three difficulty levels, and fixed random seeds.
Generate high-risk labels from a hidden probabilistic model with noise, driven by latent factors that are only partly observable, so that no feature given to the classifier fully determines the label.
Implement multi-field Jira–PR semantic alignment with the bge-base-en-v1.5 bi-encoder.
Add cross-encoder re-scoring and fine-tune the bi-encoder with Multiple Negatives Ranking Loss and hard negatives, using the training split only.
Build a calibrated alignment classifier that combines semantic, lexical, and structural features and replaces the fixed 0.7 similarity threshold with a threshold selected on validation data.
Train a leakage-free risk model: Random Forest with stratified cross-validation, a time-ordered hold-out split, and class weighting, compared with logistic regression and gradient-boosting baselines and explained with SHAP.
Evaluate alignment with ROC-AUC, PR-AUC, F1, Mean Reciprocal Rank (MRR), and Recall@k, and risk with PR-AUC, F1, and Brier score, reported per difficulty level, with ablation studies and error analysis.
Develop a modular prototype that reports alignment probability, risk probability, and the main contributing features for each PR, and document the limitations of the synthetic benchmark.

4. Scope of Work

The work will focus on synthetic Jira issue and GitHub PR pairs containing issue titles, descriptions, types, and priorities; PR titles, descriptions, branch names, code-change summaries, and changed file paths; and change metadata such as lines added and deleted, files changed, review comments,

and author activity. Code changes are represented through summaries and file paths rather than executable code.

All data will be produced by a redesigned, seeded generator (Table 2). Synthetic data is chosen because it provides exact ground truth for alignment and risk, allows the difficulty of the task to be controlled, and avoids confidentiality concerns with organizational data.

Table 2: Synthetic benchmark design

Design element	Description	Purpose
Paraphrase templates and synonyms	Several templates per ticket type (Feature, Bug, Improvement) for both Jira and PR text; synonym substitution for verbs and technical terms; varied sentence order and length	Avoid near-copy text that inflates similarity scores
Partially related negatives	PR from the same module or technology but a different task, linked to the ticket	Hard negatives that test fine-grained discrimination
Unrelated and vague negatives	PR from a different module and task, or a generic description such as "minor cleanup"	Easy negatives and a sanity check for the pipeline
Noise	Typos, missing or empty fields, extra unrelated sentences, and a small fraction of flipped labels	Realistic imperfection and robustness testing
Ground-truth labels	is_aligned column and misalignment type (none, partial, unrelated, vague)	Supervised training, threshold selection, and per-type error analysis
Hidden risk model	Latent factors (for example, code complexity, author familiarity, defect propensity) produce a risk probability through a logistic function with noise; the label is sampled from it; observed features are noisy proxies of the latent factors	Prevent label leakage and create a non-trivial learning task
Difficulty levels and seeds	Easy, medium, and hard settings that vary the share of hard negatives and noise; fixed random seeds; dataset card describing the generator	Reproducibility and robustness analysis

Alignment and risk prediction will be reported separately, and results will be broken down by difficulty level and misalignment type. Thresholds and hyperparameters will be chosen on validation data only, and the test split will not be used during model development.

Limitations will be stated explicitly in the report. Because the generator, the labels, and the models are designed by the same author, the benchmark may reflect the author's assumptions; template and synonym vocabularies are finite, so models may learn generator artefacts; the hidden risk model is a plausible assumption rather than an empirical model of software defects; and results are not expected to transfer directly to real Jira–PR data. Validation on real projects is identified as future work.

Generative LLMs, real or organizational Jira and PR data, analysis of executable code or test execution, and production deployment are out of scope. The risk score is an aid to review prioritization and is not presented as a production defect predictor.

5. Plan of Work

The work will proceed in the following phases. Exact calendar dates will be aligned with the semester schedule and institutional submission milestones.

Phase	Dates	Activities	Expected output
1. Literature review and problem definition (Weeks 1–2)	02 November 2026 – 15 November 2026	Review sentence embeddings, cross-encoders, traceability, change-risk prediction, and synthetic benchmark design; analyse limitations of the initial prototype	Problem definition and evaluation protocol
2. Synthetic benchmark design (Weeks 3–5)	16 November 2026 – 29 November 2026	Implement paraphrase templates, synonyms, partially related negatives, noise, ground-truth labels, hidden risk model, and difficulty levels; write dataset card	Versioned, seeded synthetic benchmark
3. Semantic and lexical alignment (Weeks 6–8)	30 November 2026 – 13 December 2026	Multi-field encoding with bge-base-en-v1.5; BM25, TF-IDF, and structural features; cross-encoder re-scoring	Alignment feature pipeline
4. Fine-tuning and	14 December 2026 –	Fine-tune the bi-	Trained alignment

calibrated classification (Weeks 9–11)	27 December 2026	encoder with hard negatives; train and calibrate the alignment classifier; select threshold on validation data	model
5. Risk prediction (Weeks 12–13)	28 December 2026 – 10 January 2027	Leakage-free feature set; Random Forest with cross-validation and class weighting; baselines; SHAP	Risk model with explanations
6. Evaluation, prototype, and documentation (Weeks 14–16)	10 January 2027 – 20 January 2027	Evaluate per difficulty level; ablation studies and error analysis; build reviewer-facing prototype; document limitations	Evaluation results, prototype, and final report

6. Literature References

1. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Proceedings of EMNLP-IJCNLP 2019, 3982–3992.
2. Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. Proceedings of NAACL-HLT 2019, 4171–4186.
3. Xiao, S., Liu, Z., Zhang, P., & Muennighoff, N. (2023). C-Pack: Packaged Resources To Advance General Chinese Embedding. arXiv:2309.07597.
4. Muennighoff, N., Tahmasebi, N., Magne, L., & Reimers, N. (2023). MTEB: Massive Text Embedding Benchmark. Proceedings of EACL 2023, 2014–2037.
5. Henderson, M., Al-Rfou, R., Strope, B., et al. (2017). Efficient Natural Language Response Suggestion for Smart Reply. arXiv:1705.00652.
6. Nogueira, R., & Cho, K. (2019). Passage Re-ranking with BERT. arXiv:1901.04085.
7. Robertson, S., & Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval, 3(4), 333–389.
8. Lin, J., Liu, Y., Zeng, Q., Jiang, M., & Cleland-Huang, J. (2021). Traceability Transformed: Generating More Accurate Links with Pre-Trained BERT Models. Proceedings of ICSE 2021, 324–335.
9. Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.
10. Kamei, Y., Shihab, E., Adams, B., Hassan, A. E., Mockus, A., Sinha, A., & Ubayashi, N. (2013). A Large-Scale Empirical Study of Just-in-Time Quality Assurance. IEEE Transactions on Software Engineering, 39(6), 757–773.

11. Ke, G., Meng, Q., Finley, T., et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. Advances in Neural Information Processing Systems 30 (NeurIPS 2017).
12. Lundberg, S. M., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. Advances in Neural Information Processing Systems 30 (NeurIPS 2017).
13. Niculescu-Mizil, A., & Caruana, R. (2005). Predicting Good Probabilities with Supervised Learning. Proceedings of ICML 2005, 625–632.
14. Saito, T., & Rehmsmeier, M. (2015). The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. PLoS ONE, 10(3), e0118432.
15. Kapoor, S., & Narayanan, A. (2023). Leakage and the Reproducibility Crisis in Machine-Learning-Based Science. Patterns, 4(9), 100804.
16. Wei, J., & Zou, K. (2019). EDA: Easy Data Augmentation Techniques for Boosting Performance on Text Classification Tasks. Proceedings of EMNLP-IJCNLP 2019, 6382–6388.

7. Particulars of the Supervisor and Examiner

Complete the following particulars as required by the programme. Details have been left blank rather than inferred.

Particular	Supervisor	Additional Examiner
Name	Apurva Deshpande	Ram Panchal
Qualification	BE	MCA
Designation	Software Test Engineer	Software Consultant
Employing Organization and Location	Wipro Technologies, Pune	Wipro Technologies, Pune
Phone No.(with STD Code)	8796143066	9112299931
Email Address	apurva.deshpande@wipro.com	ram.panchal@wipro.com

8. Remarks of the Supervisor

The proposed work, PRISM, addresses a relevant practical problem in software engineering: checking whether pull requests actually match their linked Jira issues, and helping reviewers decide which pull requests need closer attention. The objectives are clearly defined, achievable within the planned 14 weeks, and suitable for a WILP dissertation.

The approach adds harder, labelled synthetic data and a hidden, noisy risk model, so the classifier cannot simply relearn the labelling rule. Excluding generative language models in favor of encoder models and classical machine learning keeps the work reproducible, explainable and computationally affordable. The main limitation is that the work relies only on synthetic data.

The outline is recommended for approval.

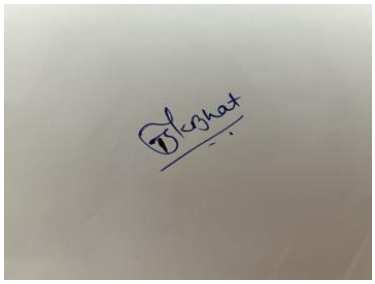
BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI

Work Integrated Learning Programmes (WILP) Division

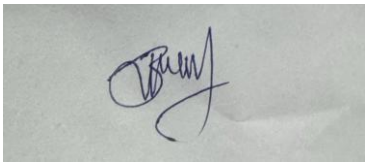
First Semester of Academic Year 2026-2027

S1-26_SSWTZG628T: Dissertation

STUDENT ID No.	2022wc15160
NAME OF THE STUDENT	Bhagyashri Kiran Bhat
STUDENT'S EMAIL ADDRESS	2022wc15160@wilp.bits-pilani.ac.in
STUDENT'S EMPLOYING ORGANIZATION & LOCATION	Wipro Technologies, Pune
SUPERVISOR'S NAME	Apurva Deshpande
SUPERVISOR'S EMPLOYING ORGANIZATION & LOCATION	Wipro Technologies, Pune
SUPERVISOR'S EMAIL ADDRESS	apurva.deshpande@wipro.com
ADDITIONAL EXAMINER'S NAME	Ram Panchal
ADDITIONAL EXAMINER'S EMPLOYING ORGANIZATION & LOCATION	Wipro Technologies, Pune
ADDITIONAL EXAMINER'S EMAIL ADDRESS	ram.panchal@wipro.com
DISSERTATION / PROJECT / PROJECT WORK TITLE	Predictive CI/CD Reliability: Predicting Workflow Failures from Pre-Build Metadata







Signature of Student	**Signature of Supervisor**	**Signature of Additional Examiner**
Name:	Name:	Name:
Bhagyashri Bhat	Apurva Deshpande	Ram Panchal